

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнила:
Студентка 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Брест 2026

Вариант 6

Цель: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Задачи лабораторной работы

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать `torchvision.datasets`). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (`matplotlib`);
2. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
6	MNIST	28X28	Adam

Ход работы

1. Конструирование и обучение СНС

Выборка MNIST содержит изображения рукописных цифр 0–9 в оттенках серого размером 28×28 пикселей: 60 000 изображений в обучающей и 10 000 в тестовой выборке. Выборка загружается средствами `torchvision.datasets.MNIST`, изображения переводятся в тензоры и нормализуются (среднее 0,1307, СКО 0,3081).

Архитектура сети построена только на базовых слоях:

- Conv2d(1→32, ядро 3×3 , padding=1) + ReLU + MaxPool2d(2) — выход $32 \times 14 \times 14$;
- Conv2d(32→64, ядро 3×3 , padding=1) + ReLU + MaxPool2d(2) — выход $64 \times 7 \times 7$;

- Flatten + Linear(3136→128) + ReLU + Dropout(0,3);
- Linear(128→10) — выход по числу классов.

Число обучаемых параметров: 421 642.

Параметры обучения: функция потерь CrossEntropyLoss, оптимизатор Adam (lr = 0.001), размер мини-батча 64, число эпох 10.

Код программы

model.py

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    """Простая свёрточная сеть для классификации MNIST (вход 1x28x28, 10 классов)."""

    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            # Блок 1: 1x28x28 -> 32x28x28 -> 32x14x14
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            # Блок 2: 32x14x14 -> 64x14x14 -> 64x7x7
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

main.py

```
"""
Лабораторная работа №1. Обучение классификаторов средствами библиотеки PyTorch.
Вариант 6: выборка MNIST, размер изображения 28x28, оптимизатор Adam.
"""

import json
import os
import random

import matplotlib.pyplot as plt
import numpy as np
import torch
```

```

import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

from model import SimpleCNN

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, "data")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
MODEL_PATH = os.path.join(BASE_DIR, "mnist_cnn.pth")

EPOCHS = 10
BATCH_SIZE = 64
LEARNING_RATE = 1e-3
SEED = 42

# Среднее и СКО яркости пикселей обучающей выборки MNIST
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

def get_loaders():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((MNIST_MEAN,), (MNIST_STD,)),
    ])
    train_set = datasets.MNIST(DATA_DIR, train=True, download=True, transform=transform)
    test_set = datasets.MNIST(DATA_DIR, train=False, download=True, transform=transform)
    train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=0)
    test_loader = DataLoader(test_set, batch_size=1000, shuffle=False, num_workers=0)
    return train_set, test_set, train_loader, test_loader

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    total_loss += loss.item() * images.size(0)

```

```

        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, 100.0 * correct / total

```

```

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        total_loss += criterion(outputs, labels).item() * images.size(0)
        correct += (outputs.argmax(1) == labels).sum().item()
        total += images.size(0)
    return total_loss / total, 100.0 * correct / total

```

```

def plot_history(history, path):
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))

    ax1.plot(epochs, history["train_loss"], "o-", label="Train Loss")
    ax1.plot(epochs, history["test_loss"], "s-", label="Test Loss")
    ax1.set_title("Ошибка (CrossEntropyLoss)")
    ax1.set_xlabel("Эпоха")
    ax1.set_ylabel("Loss")
    ax1.set_xticks(list(epochs))
    ax1.grid(alpha=0.3)
    ax1.legend()

    ax2.plot(epochs, history["train_acc"], "o-", label="Train Accuracy")
    ax2.plot(epochs, history["test_acc"], "s-", label="Test Accuracy")
    ax2.set_title("Точность, %")
    ax2.set_xlabel("Эпоха")
    ax2.set_ylabel("Accuracy, %")
    ax2.set_xticks(list(epochs))
    ax2.grid(alpha=0.3)
    ax2.legend()

    fig.suptitle("Обучение CHC на MNIST (оптимизатор Adam)")
    fig.tight_layout()
    fig.savefig(path, dpi=150)
    return fig

```

```

@torch.no_grad()
def plot_test_predictions(model, test_set, device, path, count=12):
    """Показывает случайные изображения тестовой выборки с истинным и предсказанным классом."""
    model.eval()
    indices = random.sample(range(len(test_set)), count)

```

```

fig, axes = plt.subplots(2, count // 2, figsize=(14, 6))
for ax, idx in zip(axes.flat, indices):
    image, label = test_set[idx]
    probs = torch.softmax(model(image.unsqueeze(0).to(device)), dim=1)[0]
    pred = probs.argmax().item()
    ax.imshow(image.squeeze() * MNIST_STD + MNIST_MEAN, cmap="gray")
    color = "green" if pred == label else "red"
    ax.set_title(f"True: {label}\nPred: {pred} ({probs[pred]:.1%})", color=color,
fontsize=10)
    ax.axis("off")
fig.suptitle("Примеры классификации изображений тестовой выборки")
fig.tight_layout()
fig.savefig(path, dpi=150)
return fig

def main():
    set_seed(SEED)
    os.makedirs(RESULTS_DIR, exist_ok=True)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Устройство: {device}")

    train_set, test_set, train_loader, test_loader = get_loaders()
    print(f"Обучающая выборка: {len(train_set)} изображений, тестовая: {len(test_set)}")

    model = SimpleCNN().to(device)
    print(model)
    n_params = sum(p.numel() for p in model.parameters())
    print(f"Число обучаемых параметров: {n_params}")

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

    history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}
    log_lines = []
    for epoch in range(1, EPOCHS + 1):
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer,
device)
        test_loss, test_acc = evaluate(model, test_loader, criterion, device)
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)
        line = (f"Epoch {epoch}/{EPOCHS} | Train Loss: {train_loss:.4f} | Train Acc:
{train_acc:.2f}% "
                f"| Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}%")
        print(line)
        log_lines.append(line)

    best_epoch = int(np.argmax(history["test_acc"])) + 1
    summary = (f"Итоговая точность на тестовой выборке: {history['test_acc'][-1]:.2f}% "
               f"(ошибка классификации {100 - history['test_acc'][-1]:.2f}%). ")

```

```

        f"Лучшая: {max(history['test_acc']):.2f}% на эпохе {best_epoch}")
print(summary)
log_lines.append(summary)

torch.save(model.state_dict(), MODEL_PATH)
print(f"Модель сохранена: {MODEL_PATH}")

with open(os.path.join(RESULTS_DIR, "training_log.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(log_lines) + "\n")
with open(os.path.join(RESULTS_DIR, "history.json"), "w", encoding="utf-8") as f:
    json.dump({"history": history, "params": n_params, "epochs": EPOCHS,
              "batch_size": BATCH_SIZE, "lr": LEARNING_RATE, "device": str(device)}, f,
              indent=2)

plot_history(history, os.path.join(RESULTS_DIR, "loss_accuracy.png"))
plot_test_predictions(model, test_set, device, os.path.join(RESULTS_DIR,
"test_predictions.png"))
plt.show()

if __name__ == "__main__":
    main()

```

predict.py

```

import os
import random
import sys

import matplotlib.pyplot as plt
import numpy as np
import torch
from PIL import Image, ImageOps
from torchvision import datasets

from model import SimpleCNN

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "mnist_cnn.pth")
DATA_DIR = os.path.join(BASE_DIR, "data")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
MNIST_MEAN, MNIST_STD = 0.1307, 0.3081

def choose_file():
    """Диалог выбора файла; возвращает путь или None."""
    try:
        import tkinter as tk
        from tkinter import filedialog
    except ImportError:
        return None

```

```

root = tk.Tk()
root.withdraw()
root.attributes("-topmost", True)
path = filedialog.askopenfilename(
    title="Выберите изображение цифры",
    filetypes=[("Изображения", "*.png *.jpg *.jpeg *.bmp *.gif"), ("Все файлы", "*.*")],
)
root.destroy()
return path or None

```

```

def preprocess(image):
    """Приводит произвольное изображение к формату MNIST: белая цифра на чёрном фоне 28x28,
    цифра вписана в квадрат 20x20 и расположена по центру."""
    img = image.convert("L")
    arr = np.asarray(img, dtype=np.float32)
    # В MNIST фон тёмный; если фон светлый (например, цифра на бумаге) - инвертируем
    if np.median(arr) > 127:
        img = ImageOps.invert(img)
        arr = np.asarray(img, dtype=np.float32)

    # Убираем серый фон: фон -> 0, самый яркий пиксель -> 255
    background = np.median(arr)
    arr = np.clip((arr - background) / max(arr.max() - background, 1.0), 0, 1) * 255
    img = Image.fromarray(arr.astype(np.uint8))

    # Обрезаем по границам цифры
    mask = arr > 50
    if mask.any():
        rows, cols = np.where(mask)
        img = img.crop((cols.min(), rows.min(), cols.max() + 1, rows.max() + 1))

    # Вписываем в 20x20 с сохранением пропорций и центрируем на поле 28x28
    w, h = img.size
    scale = 20.0 / max(w, h)
    img = img.resize((max(1, round(w * scale)), max(1, round(h * scale))), Image.LANCZOS)
    canvas = Image.new("L", (28, 28), 0)
    canvas.paste(img, ((28 - img.width) // 2, (28 - img.height) // 2))

    x = np.asarray(canvas, dtype=np.float32) / 255.0
    tensor = torch.from_numpy((x - MNIST_MEAN) / MNIST_STD).unsqueeze(0).unsqueeze(0)
    return tensor, x

```

```

def load_model():
    if not os.path.exists(MODEL_PATH):
        sys.exit("Файл модели не найден. Сначала запустите main.py для обучения сети.")
    model = SimpleCNN()
    model.load_state_dict(torch.load(MODEL_PATH, map_location="cpu", weights_only=True))
    model.eval()
    return model

```

```

@torch.no_grad()
def show_result(model, original, prepared, tensor, title, save_path=None):
    probs = torch.softmax(model(tensor), dim=1)[0].numpy()
    pred = int(probs.argmax())
    print(f"{title}\nПредсказанная цифра: {pred} (уверенность {probs[pred]:.2%})")
    for digit, p in enumerate(probs):
        print(f"  {digit}: {p:.4f}")

    # Карты признаков первого свёрточного слоя (после ReLU)
    feature_maps = model.features[1](model.features[0](tensor))[0]

    fig = plt.figure(figsize=(14, 5.5))
    grid = fig.add_gridspec(2, 11)

    ax = fig.add_subplot(grid[0, 0:3])
    ax.imshow(original, cmap="gray")
    ax.set_title("Исходное изображение")
    ax.axis("off")

    ax = fig.add_subplot(grid[0, 3:6])
    ax.imshow(prepared, cmap="gray")
    ax.set_title("Вход сети 28x28")
    ax.axis("off")

    ax = fig.add_subplot(grid[0, 6:11])
    colors = ["tab:green" if d == pred else "tab:blue" for d in range(10)]
    ax.bar(range(10), probs, color=colors)
    ax.set_xticks(range(10))
    ax.set_ylim(0, 1)
    ax.set_xlabel("Цифра")
    ax.set_ylabel("Вероятность")
    ax.set_title(f"Результат: {pred} ({probs[pred]:.1%})")

    for i in range(11):
        ax = fig.add_subplot(grid[1, i])
        ax.imshow(feature_maps[i].numpy(), cmap="viridis")
        ax.axis("off")
        if i == 0:
            ax.set_title("Карты признаков Conv1", loc="left", fontsize=10)

    fig.suptitle(title)
    fig.tight_layout()
    if save_path:
        fig.savefig(save_path, dpi=150)
    plt.show()

def main():

```

```

model = load_model()
path = sys.argv[1] if len(sys.argv) > 1 else choose_file()

if path:
    original = Image.open(path)
    tensor, prepared = preprocess(original)
    title = f"Файл: {os.path.basename(path)}"
    original = np.asarray(original.convert("L"))
else:
    test_set = datasets.MNIST(DATA_DIR, train=False, download=True)
    idx = random.randrange(len(test_set))
    original, label = test_set[idx]
    tensor, prepared = preprocess(original)
    title = f"Тестовое изображение MNIST №{idx}, истинная цифра: {label}"
    original = np.asarray(original)

os.makedirs(RESULTS_DIR, exist_ok=True)
name = os.path.splitext(os.path.basename(path))[0] if path else "random_test"
show_result(model, original, prepared, tensor, title,
            os.path.join(RESULTS_DIR, f"prediction_{name}.png"))

if __name__ == "__main__":
    main()

```

Результат:

Вывод программы в консоли при обучении:

```

Epoch 1/10 | Train Loss: 0.1706 | Train Acc: 94.77% | Test Loss: 0.0460 | Test Acc: 98.57%
Epoch 2/10 | Train Loss: 0.0583 | Train Acc: 98.22% | Test Loss: 0.0391 | Test Acc: 98.71%
Epoch 3/10 | Train Loss: 0.0419 | Train Acc: 98.72% | Test Loss: 0.0295 | Test Acc: 99.00%
Epoch 4/10 | Train Loss: 0.0333 | Train Acc: 98.98% | Test Loss: 0.0308 | Test Acc: 98.94%
Epoch 5/10 | Train Loss: 0.0278 | Train Acc: 99.11% | Test Loss: 0.0305 | Test Acc: 99.03%
Epoch 6/10 | Train Loss: 0.0234 | Train Acc: 99.23% | Test Loss: 0.0258 | Test Acc: 99.18%
Epoch 7/10 | Train Loss: 0.0201 | Train Acc: 99.36% | Test Loss: 0.0291 | Test Acc: 99.10%
Epoch 8/10 | Train Loss: 0.0172 | Train Acc: 99.41% | Test Loss: 0.0279 | Test Acc: 99.20%
Epoch 9/10 | Train Loss: 0.0145 | Train Acc: 99.51% | Test Loss: 0.0256 | Test Acc: 99.27%
Epoch 10/10 | Train Loss: 0.0127 | Train Acc: 99.57% | Test Loss: 0.0266 | Test Acc: 99.25%
Итоговая точность на тестовой выборке: 99.25% (ошибка классификации 0.75%). Лучшая: 99.27% на
эпохе 9

```

```

Число обучаемых параметров: 421642
Epoch 1/10 | Train Loss: 0.1706 | Train Acc: 94.77% | Test Loss: 0.0460 | Test Acc: 98.57%
Epoch 2/10 | Train Loss: 0.0583 | Train Acc: 98.22% | Test Loss: 0.0391 | Test Acc: 98.71%
Epoch 3/10 | Train Loss: 0.0419 | Train Acc: 98.72% | Test Loss: 0.0295 | Test Acc: 99.00%
Epoch 4/10 | Train Loss: 0.0333 | Train Acc: 98.98% | Test Loss: 0.0308 | Test Acc: 98.94%
Epoch 5/10 | Train Loss: 0.0278 | Train Acc: 99.11% | Test Loss: 0.0305 | Test Acc: 99.03%
Epoch 6/10 | Train Loss: 0.0234 | Train Acc: 99.23% | Test Loss: 0.0258 | Test Acc: 99.18%
Epoch 7/10 | Train Loss: 0.0201 | Train Acc: 99.36% | Test Loss: 0.0291 | Test Acc: 99.10%
Epoch 8/10 | Train Loss: 0.0172 | Train Acc: 99.41% | Test Loss: 0.0279 | Test Acc: 99.20%
Epoch 9/10 | Train Loss: 0.0145 | Train Acc: 99.51% | Test Loss: 0.0256 | Test Acc: 99.27%
Epoch 10/10 | Train Loss: 0.0127 | Train Acc: 99.57% | Test Loss: 0.0266 | Test Acc: 99.25%
Итоговая точность на тестовой выборке: 99.25% (ошибка классификации 0.75%). Лучшая: 99.27% на эпохе 9

```

Обучение CNN на MNIST (оптимизатор Adam)

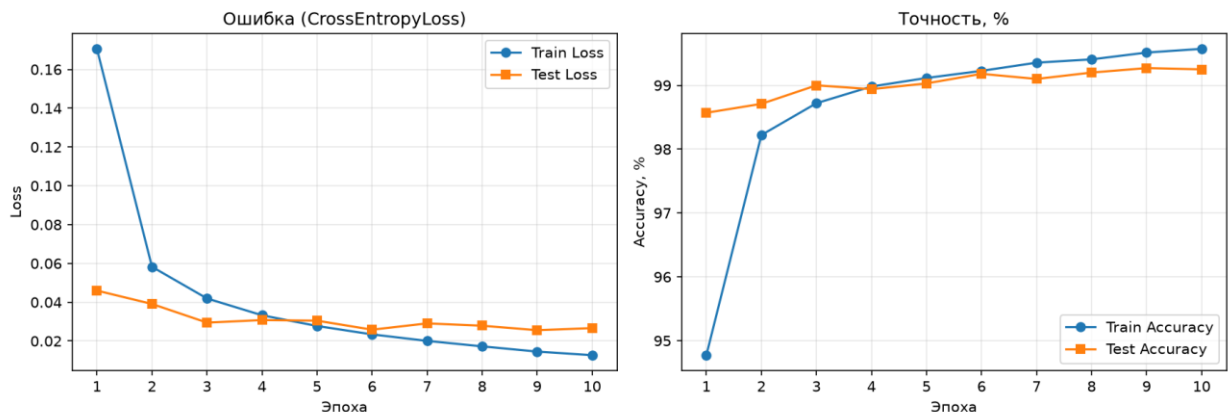


Рисунок 1 – Графики изменения ошибки и точности на обучающей и тестовой выборках

После 10 эпох точность на тестовой выборке составила 99.25% (ошибка классификации 0.75%, т.е. около 75 неверно распознанных изображений из 10 000), значение функции потерь на тесте — 0.0266. Наилучшая точность 99.27% достигнута на эпохе 9. Уже после первой эпохи точность на тесте составила 98.57%, что показывает быструю сходимость оптимизатора Adam.

Примеры классификации изображений тестовой выборки

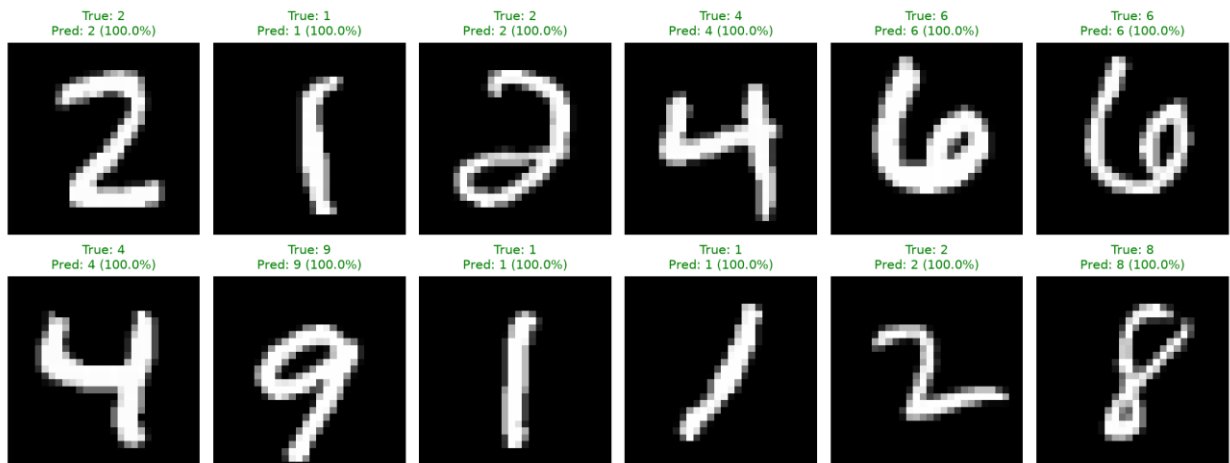


Рисунок 2 – Классификация случайных изображений тестовой выборки

2. Сравнение с state-of-the-art результатами

Модель	Точность, %	Ошибка, %
Ансамбль СНС с гомогенными векторными капсулами (Byerly et al., 2021)	99,87	0,13
Ансамбль СНС с DropConnect (Wan et al., 2013)	99,79	0,21
Многоколоночная глубокая СНС MCDNN (Cireşan et al., 2012)	99,77	0,23
LeNet-5 (LeCun et al., 1998)	99,05	0,95
Собственная СНС (данная работа)	99,25	0,75

Полученная точность 99.25% лишь на 0.62 процентного пункта уступает лучшему известному результату на MNIST и превосходит классическую сеть LeNet-5. При этом разработанная сеть содержит всего два свёрточных слоя, обучалась 10 эпох и не использовала аугментацию данных, пакетную нормализацию или ансамблирование. State-of-the-art модели достигают ошибки 0,13–0,23% за счёт ансамблей из нескольких сетей, аугментации (сдвиги, повороты, эластичные искажения), пакетной нормализации и длительного обучения.

Из графиков видно, что ошибка на обучающей выборке монотонно снижается на всём протяжении обучения, а ошибка на тестовой выборке после 6-й эпохи выходит на плато (около 0,026–0,029), и точность на тесте стабилизируется на уровне 99,1–99,3%. Начиная с 5-й эпохи ошибка на обучающей выборке становится ниже тестовой — это признак начала переобучения, поэтому дальнейшее увеличение числа эпох без дополнительной регуляризации не даст заметного прироста точности. Выборка MNIST относительно проста: основная часть ошибок сети приходится на неоднозначно или небрежно написанные цифры, которые затруднительно распознать и человеку.

3. Визуализация работы СНС

Для визуализации реализован скрипт predict.py. При запуске открывается окно выбора файла, в котором можно выбрать произвольное изображение цифры (png, jpg, bmp). Изображение переводится в оттенки серого, при светлом фоне инвертируется (в MNIST цифры белые на чёрном фоне), обрезается по границам цифры, вписывается в квадрат 20×20 и центрируется на поле 28×28 — так же, как подготовлены изображения MNIST. Затем изображение подаётся на вход обученной сети; выводятся исходное изображение, вход сети, вероятности классов и карты признаков первого свёрточного слоя. Если файл не выбран, берётся случайное изображение тестовой выборки.

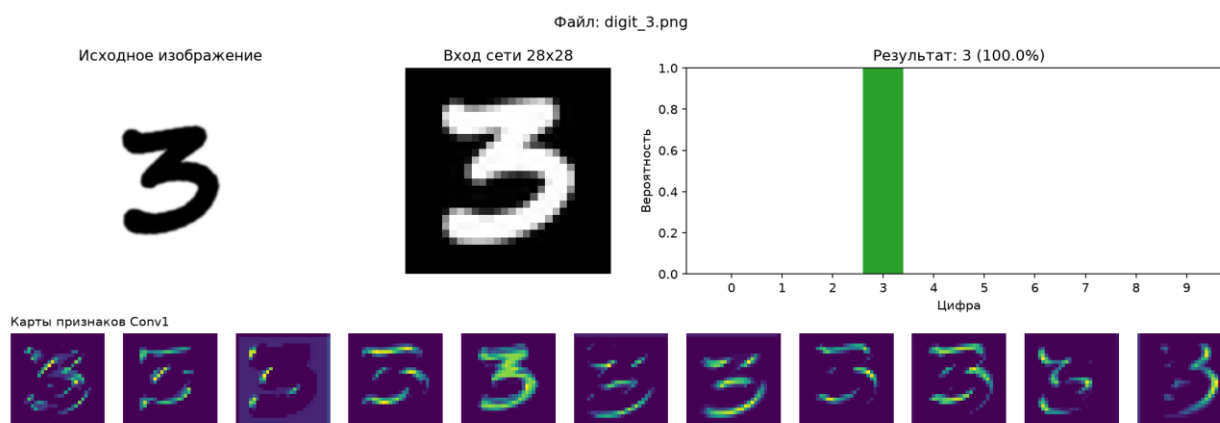


Рисунок 3 – Результат распознавания (digit_3)

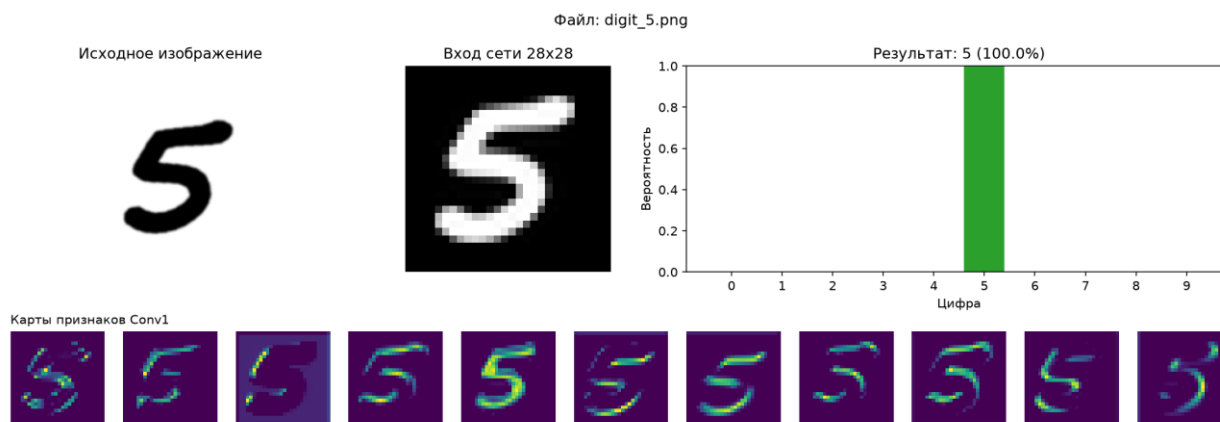


Рисунок 4 – Результат распознавания (digit_5)

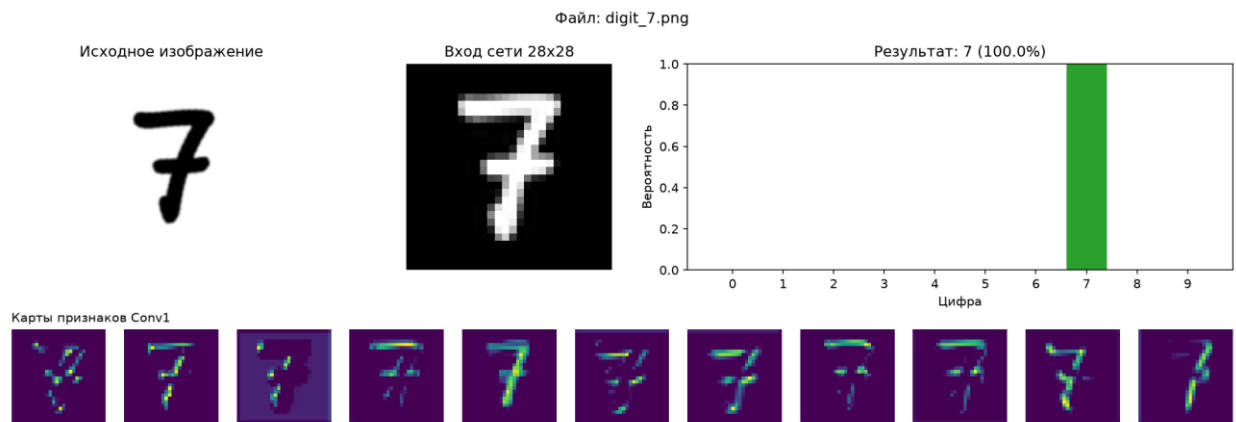


Рисунок 5 – Результат распознавания (digit_7)

Вывод программы predict.py в консоли:

Файл: digit_3.png

Предсказанная цифра: 3 (уверенность 100.00%)

```
0: 0.0000
1: 0.0000
2: 0.0000
3: 1.0000
4: 0.0000
5: 0.0000
6: 0.0000
7: 0.0000
8: 0.0000
9: 0.0000
```

Файл: digit_5.png

Предсказанная цифра: 5 (уверенность 100.00%)

```
0: 0.0000
1: 0.0000
2: 0.0000
3: 0.0000
4: 0.0000
5: 1.0000
6: 0.0000
7: 0.0000
8: 0.0000
9: 0.0000
```

Файл: digit_7.png

Предсказанная цифра: 7 (уверенность 99.96%)

```
0: 0.0000
1: 0.0000
2: 0.0001
3: 0.0003
4: 0.0000
5: 0.0000
6: 0.0000
7: 0.9996
```

8: 0.0000

9: 0.0000

Свёрточная нейронная сеть даже простой архитектуры из двух свёрточных слоёв, слоёв подвыборки, нелинейности ReLU и двух полносвязных слоёв успешно извлекает признаки рукописных цифр (штрихи, изгибы, пересечения линий) и уверенно их классифицирует. Оптимизатор Adam за счёт адаптивного шага обучения обеспечил быструю сходимость: высокая точность достигается уже за первые эпохи.

Ограничения такой сети проявляются на изображениях, которые отличаются от обучающей выборки (другая толщина линий, стиль написания, положение цифры), поэтому для произвольных изображений важна предобработка, приводящая их к формату MNIST. Для приближения к state-of-the-art результатам необходимы аугментация данных, пакетная нормализация, более глубокие архитектуры и ансамбли моделей.

Вывод: проделав данную лабораторную работу, я научилась конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения средствами библиотеки PyTorch. Разработанная СНС, обученная на выборке MNIST с оптимизатором Adam, достигла точности 99.25% на тестовой выборке.